Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2026**

Student: Riccardo Polo Discussed with: FULL NAME

Report for Project 1

Due date: 09 March 2026, 18:00 (Europe/Zurich)

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
    - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. Euler warm-up [10 points]

Question 1: What is the module system, and how is it used?
The module system (Lmod, a Lua-based environment modules implementation) is used on HPC systems to dynamically manage software environments. It allows users to load, unload, and switch between different software versions without manually modifying system paths.
Loading all software globally would lead to conflicts between different versions of compilers and libraries. The module system avoids this problem by dynamically modifying environment variables.
In practice, before compiling or running programs on Euler, users load the required modules (e.g., compilers or libraries) inside their batch scripts using commands such as `module load`.

Question 2: What is Slurm, and what is its purpose?
Slurm is an open-source workload manager and job scheduler for Linux and Unix-like systems, widely used by supercomputers and clusters. It schedules and executes jobs on compute nodes, handling resource allocation, queuing, and execution control.
In particular (from the Slurm documentation), it provides three main functions:

- it allocates exclusive and/or non-exclusive access to resources (compute nodes, CPUs, memory, GPUs) to users for a specified duration of time so they can perform work;
- it provides a framework for starting, executing, and monitoring work (normally a parallel job) on the set of allocated nodes.
- it arbitrates contention for resources by managing a queue of pending jobs.

We now illustrate how the remaning questions were adressed.
The solution to question 3 and 4 can be found by first running the Makefile provided in the directory `1_Euler_Warmup` and then running the file `slurm_job_one.sh`. We already provide in the directory `1_Euler_Warmup/build_job_one` the ouputs of the batch script as requested. The output is reported also here for completeness:

```
Currently Loaded Modules:
1) stack/2025-06 2) gcc/12.2.0



Model name: AMD EPYC 9654 96-Core Processor
eu-g9-101-4
Hello World from eu-g9-101-4
```

As we can see we have specified to use a node with AMD EPYC 9654 96-Core Processor, and the job was executed on the node `eu-g9-101-4`.
The solution to question 5 can be found by running the file `slurm_job_two.sh`. The output of the batch script is reported here for completeness:

```
Currently Loaded Modules:
  1) stack/2025-06 2) gcc/12.2.0



Model name: AMD EPYC 7763 64-Core Processor
Model name: AMD EPYC 7763 64-Core Processor
eu-a2p-487
eu-a2p-482
Hello World from eu-a2p-487
Hello World from eu-a2p-482
```

We have thus successfully executed the same job on two nodes, both with AMD EPYC 7763 64-Core Processor, and jobs were executed on the nodes `eu-a2p-487` and `eu-a2p-482` respectively.

2. Performance characteristics [50 points]

2.1. Peak performance [16 Points]

Proceeding as in the example given in the project description, the hardware specifications can be obtained from the Euler webpage.
For Euler VII (phase 1), the system consists of $n_{\text{nodes}} = 292$ nodes, each equipped with $n_{\text{sockets}} = 2$ sockets and $n_{\text{cores}} = 64$ cores per socket, operating at a frequency $f = 2.6\,\text{GHz}$.
For Euler VII (phase 2), the system consists of $n_{\text{nodes}} = 248$ nodes, each with $n_{\text{sockets}} = 2$ sockets and $n_{\text{cores}} = 64$ cores per socket, operating at a frequency $f = 2.45\,\text{GHz}$.

Using the uops.info database provided, we determine both the superscalability factor and the FMA factor for the corresponding AMD architectures.
For both Euler VII phases, which are based on AMD Zen2 and AMD Zen3 respectively, we obtain $n_{\text{super}} = 2$ and $n_{\text{FMA}} = 2$.

The FMA factor is determined by observing that all instructions marked as FMA in the table have a corresponding number of micro-operations equal to one. This indicates that the fused multiply-add is executed as a single micro-operation by dedicated hardware. Since an FMA instruction performs the operation $a \cdot b + c$, it consists of one floating-point multiplication and one floating-point addition. Therefore, each FMA instruction corresponds to two floating-point operations, and thus $n_{\text{FMA}} = 2$.

The superscalability is defined as the number of floating-point operations that the FPU of a core can execute in parallel within a single clock cycle. To determine this value, we analyze the instruction `VFMADD132PD (YMM, YMM, YMM)` reported in the uops.info table. The table specifies the throughput 0.50 cycles/instruction implies 2 vector FMA instructions can be issued per cycle. Since the table indicates execution on FP0/1, we conclude the core can sustain two FMA pipelines, hence $n_{\text{super}} = 2$. The factor $n_{\text{FMA}} = 2$ then accounts for mul+add inside FMA a throughput of 0.50 cycles per instruction, one micro-operation, and execution on ports FP0/1.

Hence, for both AMD Zen 2 and AMD Zen 3, the superscalability is 2 floating-point operations per cycle per core.

To determine n_{SIMD}, we inspect the floating-point instructions reported in the uops.info table, where instructions such as `VFMADD132PD (YMM, YMM, YMM)` appear. The presence of YMM operands indicates the use of 256-bit AVX registers, since YMM registers are defined as 256-bit wide in the AVX/AVX2 instruction set. As both AMD Zen 2 and Zen 3 execute these YMM-based instructions, they necessarily support AVX2 with 256-bit SIMD registers. A double-precision floating-point number occupies 64 bits; therefore, the number of doubles that can be processed concurrently by one SIMD instruction is

$$n_{\text{SIMD}} = \frac{256}{64} = 4.$$

Hence, each SIMD instruction can operate on four double-precision values in parallel.

Parameter	**Euler VII (Phase 1)**	**Euler VII (Phase 2)**
n_{nodes}	292	248
n_{sockets}	2	2
n_{cores} (per socket)	64	64
Clock frequency f	2.6 GHz	2.45 GHz
n_{FMA}	2	2
n_{super} (FMA instructions per cycle per core)	2	2
n_{SIMD} (doubles per SIMD instr.)	4	4

Table 1: Summary of hardware parameters and architectural factors used for peak performance estimation.

We can now compute all the peak performance parameters for both Euler VII phases :

$$P_{\text{core},1} = n_{\text{super}} \times n_{\text{FMA}} \times n_{\text{SIMD}} \times f = 2 \times 2 \times 4 \times 2.6\,\text{GHz} = 41.6\,\text{GFLOPS/core}.$$
$$P_{\text{CPU},1} = n_{\text{cores}} \times P_{\text{core},1} = 64 \times 41.6\,\text{GFLOPS/core} = 2.6624\,\text{TFLOPS/CPU}.$$
$$P_{\text{node},1} = n_{\text{sockets}} \times P_{\text{CPU},1} = 2 \times 2.6624\,\text{TFLOPS/node} = 5.3248\,\text{TFLOPS/node}.$$
$$P_{\text{cluster},1} = n_{\text{nodes}} \times P_{\text{node},1} = 292 \times 5.3248\,\text{TFLOPS/node} = 1555.04\,\text{TFLOPS}.$$

$$P_{\text{core},2} = n_{\text{super}} \times n_{\text{FMA}} \times n_{\text{SIMD}} \times f = 2 \times 2 \times 4 \times 2.45\,\text{GHz} = 39.2\,\text{GFLOPS/core}.$$
$$P_{\text{CPU},2} = n_{\text{cores}} \times P_{\text{core},2} = 64 \times 39.2\,\text{GFLOPS/core} = 2.5088\,\text{TFLOPS/CPU}.$$
$$P_{\text{node},2} = n_{\text{sockets}} \times P_{\text{CPU},2} = 2 \times 2.5088\,\text{TFLOPS/node} = 5.0176\,\text{TFLOPS/node}.$$
$$P_{\text{cluster},2} = n_{\text{nodes}} \times P_{\text{node},2} = 248 \times 5.0176\,\text{TFLOPS/node} = 1244.3648\,\text{TFLOPS}.$$

2.2. Memory Hierarchies [8 Points]

2.2.1. Cache and main memory size

The solution to this exercise can be found by first running the Makefile in `2_Performance_Characteristics/Memory_Hierarchies` and then executing `memory_hierarchies_01.sh` for Euler VII (phase 1). The batch-script output is available in `2_Performance_Characteristics/Memory_Hierarchies/build_1`.
We report in table 2 the results for Euler VII (phase 1) and in figure 1 the memory topology.

Main memory (total)	249 GB
L3 cache (total)	512 MB (32 × 16 MB)
L3 cache (per socket)	256 MB (16 × 16 MB)
L2 cache (per core)	512 KB
L1 data cache (per core)	32 KB
L1 instruction cache (per core)	32 KB

Table 2: Memory hierarchy of an Euler compute node with AMD EPYC 7H12 (2 sockets, 64 cores per socket). Each core has private L1 and L2 caches, while L3 cache is organized in 32 blocks of 16 MB shared among groups of 4 cores.

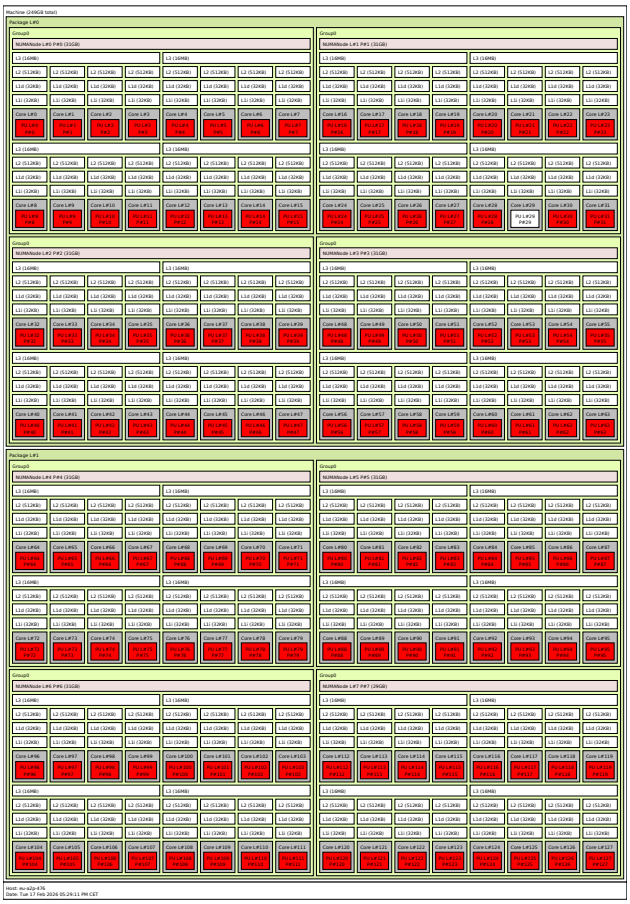


Figure 1: Topology of Euler VII (phase 1) compute node with AMD EPYC 7H12 (2 sockets, 64 cores per socket).

We proceed in the same way for Euler VII (phase 2) by executing `memory_hierarchies_02.sh` and we report in table 3 the results for Euler VII (phase 2) and in figure 2 the memory topology.

Main memory (total)	249 GB
L3 cache (total)	512 MB (16 × 32 MB)
L3 cache (per socket)	256 MB (8 × 32 MB)
L2 cache (per core)	512 KB
L1 data cache (per core)	32 KB
L1 instruction cache (per core)	32 KB

Table 3: Memory hierarchy of an Euler compute node with AMD EPYC 7763 (2 sockets, 64 cores per socket). Each core has private L1 and L2 caches, while the L3 cache is organized in 16 blocks of 32 MB shared among groups of cores.

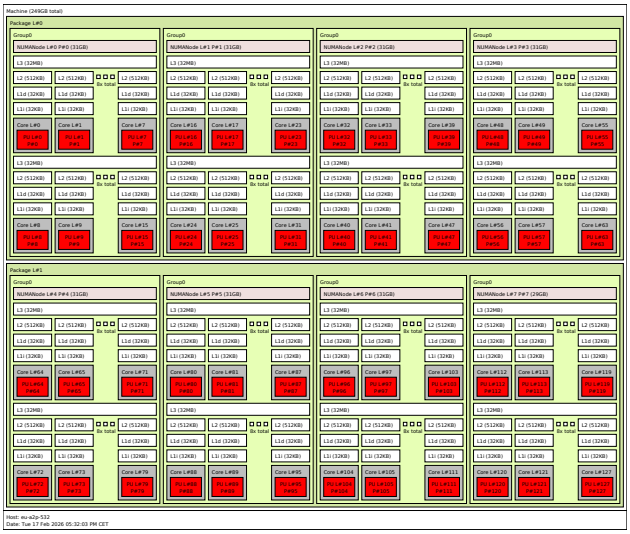


Figure 2: Topology of an Euler compute node with AMD EPYC 7763 (2 sockets, 64 cores per socket).

2.3. Bandwidth: STREAM benchmark [10 Points]

For this exercise one can run the Makefile in `2_Performance_Characteristics/STREAM_Benchmark` and then execute `stream_one.sh` for Euler VII (phase 1) and `stream_two.sh` for Euler VII (phase 2). The batch-script outputs are available in `2_Performance_Characteristics/STREAM_Benchmark/build_1` and `2_Performance_Characteristics/STREAM_Benchmark/build_2` respectively.

For Euler VII (phase 1), we run the file stream.c with the parameters specified in the following code snippet, which corresponds to the command used in the batch script `stream_one.sh`:

```
gcc -O3 -march=native -mcmodel=medium -DSTREAM_TYPE=double -DSTREAM_ARRAY_SIZE=268435456
    -DNTIMES=20 stream.c -o build_1/stream_one_c.exe
```

We used as array size the value of 2^{28}, which corresponds to 2 GB per array, and we set the number of iterations to 20. The corresponding output of the batch script is reported here for completeness:

```
  Currently Loaded Modules:
  1) stack/2025-06 2) gcc/12.2.0




Model name:  AMD EPYC 7H12 64-Core Processor
eu-a2p-476
-------------------------------------------------------------
STREAM version $Revision: 5.10 $
```

```
-------------------------------------------------------------
This system uses 8 bytes per array element.
-------------------------------------------------------------
Array size = 268435456 (elements), Offset = 0 (elements)
Memory per array = 2048.0 MiB (= 2.0 GiB).
Total memory required = 6144.0 MiB (= 6.0 GiB).
Each kernel will be executed 20 times.
 The *best* time for each kernel (excluding the first iteration)
 will be used to compute the reported bandwidth.
-------------------------------------------------------------
Your clock granularity/precision appears to be 1 microseconds.
Each test below will take on the order of 151041 microseconds.
   (= 151041 clock ticks)
Increase the size of the arrays if this shows that
you are not getting at least 20 clock ticks per test.
-------------------------------------------------------------
WARNING -- The above is only a rough guideline.
For best results, please be sure you know the
precision of your system timer.
-------------------------------------------------------------
Function Best Rate MB/s Avg time Min time Max time
Copy: 33537.1 0.133024 0.128066 0.146805
Scale: 18170.2 0.246928 0.236374 0.258205
Add: 20230.7 0.332126 0.318450 0.354032
Triad: 19668.1 0.341913 0.327558 0.364689
-------------------------------------------------------------
Solution Validates: avg error less than 1.000000e-13 on all three arrays
-------------------------------------------------------------
```

We choose `STREAM_ARRAY_SIZE` $= 2^{28}$ (268,435,456 doubles) so that each array occupies 2 GiB and the total working set (3 arrays $\approx$ 6 GiB) is well beyond the last-level cache. This reduces cache effects and makes the reported rates representative of sustained main-memory bandwidth. STREAM reports the *Best Rate* excluding the first iteration; for the roofline model we use the Triad kernel as a proxy for sustainable bandwidth, since it stresses both loads and stores and is commonly the most representative. The Copy kernel can appear faster because it benefits from efficient store handling (e.g., write combining), whereas Add/Triad are more load-intensive. Here, Triad achieves 19.7 GB/s (19,668 MB/s), which we use to define B_1 (after converting MB/s to GB/s).

For Euler VII (phase 2), we run the file stream.c with the parameters specified in the following code snippet, which corresponds to the command used in the batch script `stream_two.sh`:

```
gcc -O3 -march=native -mcmodel=medium -DSTREAM_TYPE=double -DSTREAM_ARRAY_SIZE=268435456
    -DNTIMES=20 stream.c -o build_2/stream_two_c.exe
```

Here instead we used as array size the value of 2^{28}, which corresponds to 160 MB per array, and we set the number of iterations to 20. The corresponding output of the batch script is reported here for completeness:

```
Currently Loaded Modules:
  1) stack/2025-06 2) gcc/12.2.0



Model name: AMD EPYC 7763 64-Core Processor
eu-a2p-509
-------------------------------------------------------------
STREAM version $Revision: 5.10 $
-------------------------------------------------------------
This system uses 8 bytes per array element.
-------------------------------------------------------------
Array size = 268435456 (elements), Offset = 0 (elements)
```

```
Memory per array = 2048.0 MiB (= 2.0 GiB).
Total memory required = 6144.0 MiB (= 6.0 GiB).
Each kernel will be executed 20 times.
 The *best* time for each kernel (excluding the first iteration)
 will be used to compute the reported bandwidth.
-------------------------------------------------------------
Your clock granularity/precision appears to be 1 microseconds.
Each test below will take on the order of 206516 microseconds.
   (= 206516 clock ticks)
Increase the size of the arrays if this shows that
you are not getting at least 20 clock ticks per test.
-------------------------------------------------------------
WARNING -- The above is only a rough guideline.
For best results, please be sure you know the
precision of your system timer.
-------------------------------------------------------------
Function Best Rate MB/s Avg time Min time Max time
Copy: 15183.6 0.291832 0.282868 0.298069
Scale: 13359.4 0.326391 0.321494 0.331896
Add: 12655.6 0.515276 0.509060 0.519799
Triad: 14198.6 0.468732 0.453738 0.498993
-------------------------------------------------------------
Solution Validates: avg error less than 1.000000e-13 on all three arrays
-------------------------------------------------------------
```

Note that `STREAM_ARRAY_SIZE`= 268435456 with `STREAM_TYPE=double` corresponds to $268435456 \cdot 8 = 2^{28} \cdot 8 = 2^{31}$ bytes, i.e. 2.0,GiB per array (as confirmed by STREAM: 2048.0,MiB). Therefore the total working set is $3 \times 2.0, \text{GiB} \approx 6.0, \text{GiB}$, which is well beyond the last-level cache capacity and strongly reduces cache effects. Hence, the reported bandwidth numbers are representative of sustained main-memory (DRAM) bandwidth. As for the roofline model, we use the `Triad` kernel as a proxy for sustainable bandwidth; in this run, `Triad` achieves 14198.6,MB/s (≈ 14.20,GB/s), which we take as B_2.

2.4. Performance model: A simple roofline model [16 Points]

A (naive) roofline model provides an upper bound on the attainable performance of a kernel as a function of its operational intensity I (in Flops/Byte). For a given hardware, the attainable performance is bounded by:

- a **compute ceiling** given by the peak floating-point performance P_{peak} (in GFlops/s),
- a **bandwidth ceiling** given by $B \cdot I$, where B is the sustainable memory bandwidth (in GBytes/s).

Hence the roofline bound is

$$P(I) = \min\left(P_{\text{peak}},\ B \cdot I\right).$$

The transition between memory-bound and compute-bound regimes occurs at the **ridge point**:

$$I_{\text{ridge}} = \frac{P_{\text{peak}}}{B}. \tag{1}$$

A kernel with $I < I_{\text{ridge}}$ is memory-bound, while a kernel with $I > I_{\text{ridge}}$ is compute-bound.

From the previous exercises, we have already computed the peak performance P_{peak} and the sustainable memory bandwidth B for both Euler VII phases. The values of the single core peak performances where

$$P_{\text{peak},1} = 41.6 \text{ GFlops/s} \quad \text{(Euler VII phase 1, AMD EPYC 7H12 @ 2.6 GHz)},$$
$$P_{\text{peak},2} = 39.2 \text{ GFlops/s} \quad \text{(Euler VII phase 2, AMD EPYC 7763 @ 2.45 GHz)}.$$

We use the STREAM `Triad` bandwidth as sustainable memory bandwidth, since it represents a typical streaming workload (three arrays, two loads + one store per iteration) and is commonly used in roofline constructions. From our STREAM outputs, we have the following sustainable memory bandwidths:

$$B_1 = 19668.1 \text{ MB/s} = 19.6681 \text{ GB/s},$$
$$B_2 = 149198.6 \text{ MB/s} = 14.2 \text{ GB/s},$$

where we convert MB/s to GB/s by dividing by 1000.
Using Eq.1:

$$I_{\text{ridge},1} = \frac{P_{\text{peak},1}}{B_1} = \frac{41.6}{19.6681} \approx 2.12 \text{ Flops/Byte},$$
$$I_{\text{ridge},2} = \frac{P_{\text{peak},2}}{B_2} = \frac{39.2}{14.2} \approx 2.76 \text{ Flops/Byte}.$$

Quantity	**Euler VII (Phase 1)**	**Euler VII (Phase 2)**
P_{peak} [GFlops/s]	41.6	39.2
B (STREAM Triad) [GB/s]	19.6681	14.20
$I_{\text{ridge}} = P_{\text{peak}}/B$ [Flops/Byte]	2.12	2.76

Table 4: Inputs and ridge point of the single-core roofline models.

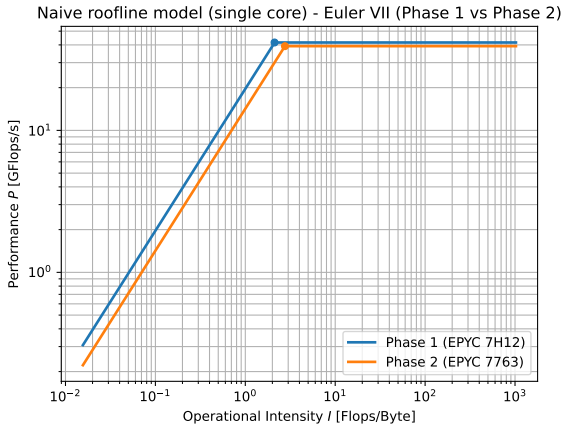


Figure 3: Naive roofline model for a single core on Euler VII (both phases). The ridge points are at $I_{\text{ridge},1} \approx 2.12$ Flops/Byte and $I_{\text{ridge},2} \approx 2.76$ Flops/Byte for phase 1 and phase 2 respectively.

3. Auto-vectorization [10 points]

Question 1: what is data alignment and is it important for data structures to be aligned?
Data alignment refers to placing an object in memory at an address that is a multiple of some power-of-two boundary (for example, 16 bytes). In the document, alignment is discussed as a key performance property for SIMD loads/stores: Intel SSE (and similarly other SIMD ISAs) prefers aligned memory references, and data movement instructions can be significantly more efficient when the data is aligned. The text also warns that using aligned data-movement instructions with incorrectly aligned addresses can even raise an exception (so alignment assertions must be correct).
Because of this, it is generally important that frequently accessed arrays/structures used in vectorized loops are aligned (e.g., 16-byte boundaries for SSE), especially when you want the compiler to generate efficient aligned vector loads/stores and avoid extra fix-up code.

Question 2: what are some obstacles that can prevent automatic vectorization by the compiler?
Several recurring patterns can make vectorization unsafe or not worthwhile, so the compiler may decide to keep scalar code. A first frequent issue is **non-contiguous memory access**: when a loop uses non-unit strides or indirect addressing (for instance, indices coming from another array), the hardware cannot easily use a single packed load, and the compiler anticipates extra gathers and inefficient memory traffic.
Another common blocker is the presence of **loop-carried dependencies**. Since SIMD executes multiple iterations "together", it effectively changes the per-iteration execution order; if the compiler cannot prove that different iterations are independent, it must conservatively assume a dependence and will often avoid vectorization.
Vectorization is also discouraged when the loop is not in a **canonical vector-friendly form**. Compilers prefer countable loops whose trip count is known at entry, with a single entry and a single exit, and with straight-line bodies; complex or unsupported control flow (e.g. `switch` statements, early exits, or other non-structured patterns) makes it harder to build a correct vector loop. Moreover, **function calls inside the loop body** frequently stop vectorization: even something as simple as printing introduces side effects and prevents reordering, unless the call is a recognized intrinsic, can be proven safe, or can be inlined.
Finally, **pointer aliasing** and complicated pointer expressions can force the compiler into worst-case assumptions: if two pointers might refer to overlapping memory, then loads and stores may depend on each other across iterations, and the compiler may refuse vectorization or add costly runtime checks.

Question 3: is there a way to help the compiler to vectorize and how?
Yes. One can increase the chances of vectorization by writing vector-friendly code and by supplying additional information that the compiler cannot safely infer. A direct way is to use **pragmas** that communicate dependence and vectorization intent: for instance, `#pragma ivdep` requests the compiler to ignore assumed dependencies, while directives of the form `#pragma vector ...` can specify whether the vector loop should be generated and how alignment should be treated, potentially overriding conservative profitability heuristics.
A second powerful aid is to reduce aliasing uncertainty with `restrict` (C99). When `restrict` is correct, it asserts that pointers do not overlap, so the compiler can remove conservative dependence assumptions and avoid inserting runtime alias checks. In the same spirit, the user can **communicate alignment** properties: data can be declared with explicit alignment (e.g. `__declspec(align(...))`), and compilers also provide mechanisms to assume alignment or other conditions (e.g. `__builtin_assume_aligned(a,n)` or more general assumptions via `__builtin_assume(cond)`), which may unlock aligned vector loads and reduce peeling/checking overhead. Finally, **data layout choices** matter: adopting a structure-of-arrays (SoA) layout often turns accesses into unit-stride streams, which are typically much easier and more profitable to vectorize than the scattered accesses typical of an array-of-structures (AoS) layout.

Question 4: which loop optimizations are performed by the compiler to vectorize and pipeline loops?
Successful vectorization is often enabled only after stronger optimization settings activate additional loop transformations that reshape the loop into a more regular form. In this context, the compiler commonly applies **loop unrolling** together with packed SIMD instruction generation: vectorization produces SIMD instructions that cover multiple scalar iterations, which is conceptually similar to unrolling, and manual unrolling is generally unnecessary if its only purpose is to "force" vectorization.
Another important transformation is **loop peeling for alignment**. By executing a small number of scalar iterations first, the compiler can reach an aligned address boundary (e.g. 16-byte alignment for SSE) so that the main vector loop can use aligned accesses more consistently and efficiently. Since vectorization typically

targets the *innermost* loop, earlier loop transformations such as **interchange, collapsing, or related restructuring effects** can change which loop becomes innermost and thus becomes the vectorization target. Overall, the compiler tries to transform the original code into a regular, countable, straight-line innermost loop and, when possible, it adds alignment-oriented restructuring so that the vectorized and pipelined version runs efficiently.

Question 5: what can be done if automatic vectorization by the compiler fails or is suboptimal?
A robust approach is to start from diagnosis, then remove the cause through code restructuring, and only afterwards apply stronger directives or explicit SIMD methods. Practically, the first step is to consult **vectorization reports and diagnostics** to identify whether the loop was rejected due to dependencies, aliasing, control flow, memory access patterns, or because the compiler predicted low benefit. Based on that feedback, one can often **rewrite the loop into a more vectorizable shape**: make the trip count well-defined, keep a single entry/exit, avoid unsupported control flow, remove or move function calls out of the loop, eliminate loop-carried dependencies, and prefer unit-stride array accesses over indirect indexing or complex pointer arithmetic.
When correctness allows it, **adding accurate annotations** can resolve conservative assumptions: `restrict` can remove aliasing ambiguity, alignment declarations or assumptions can reduce misalignment penalties, and pragmas such as `#pragma ivdep` or `#pragma vector always` can steer the compiler past overly cautious dependence or profitability decisions. These mechanisms must be used carefully, since forcing aligned loads on misaligned addresses or providing incorrect dependence information can change semantics or even lead to crashes. If hints remain insufficient, **user-mandated SIMD directives** such as `#pragma omp simd` can be used to impose vectorization semantics and typically trigger warnings if the compiler cannot generate a vector loop, while also allowing clauses to specify the intended SIMD behavior. Finally, enabling broader optimization context—for instance with **high optimization levels and interprocedural optimization**—can improve vectorization by exposing more information (e.g. enabling inlining and better dependence reasoning) and by applying additional transformations. If performance is still unsatisfactory, the last step is to move to **more explicit SIMD programming** (e.g. SIMD extensions or intrinsics), which provides tighter control and reduces reliance on compiler heuristics.

4. Matrix multiplication optimization [30 points]

Task 1: Improve the loop ordering in `dgemm_jki.c`.
The original implementation uses the loop order i–j–k:

```
void square_dgemm(int n, double* A, double* B, double* C) {
  for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
      for(int k = 0; k < n; ++k) {
        C[i+j*n] += A[i+k*n] * B[k+j*n];
      }
    }
  }
}
```

This loop ordering is correct, but it performs poorly for column-major storage (indexing $i + jn$). In the innermost loop over k, the access to $A[i + kn]$ moves across different columns of A with a stride of n, i.e., it repeatedly jumps to memory locations that are far apart. This limits spatial locality and makes it difficult for the cache hierarchy and hardware prefetchers to be effective. By contrast, $B[k+jn]$ is read contiguously in the k-loop (a whole column of B), so the main bottleneck of the naive ordering is the strided access pattern to A.

To improve locality, we reorder the loops to j–k–i and hoist invariant quantities out of the innermost loop:

```
void square_dgemm(int n, double* A, double* B, double* C) {
  for (int j = 0; j < n; ++j) {
    double* Ccol = C + j*n; // C(:,j)
    for (int k = 0; k < n; ++k) {
      const double bkj = B[k + j*n]; // B(k,j) reused for all i
      const double* Acol = A + k*n; // A(:,k)
      for (int i = 0; i < n; ++i) {
        Ccol[i] += Acol[i] * bkj; // stride-one on C and A
      }
    }
  }
}
```

With column-major layout, full columns are contiguous in memory. For fixed (j, k), the innermost loop over i streams through $A(:, k)$ and updates $C(:, j)$ with *stride 1* (contiguous memory) in both arrays. This improves cache utilization and makes prefetching more effective, since consecutive iterations access consecutive memory locations. Moreover, loading $B(k, j)$ once into a scalar (`bkj`) and using column pointers (`Acol`, `Ccol`) reduces redundant address computations and avoids repeatedly reloading the same element of B. The computation is unchanged, since we still perform

$$C_{i,j} \mathrel{+}= \sum_{k=0}^{n-1} A_{i,k} B_{k,j},$$

but with a memory-access pattern that is more favorable for the cache hierarchy and that typically facilitates SIMD vectorization in the innermost loop.

Task 2: Implement blocking for square matrix multiplication in `dgemm_blocked.c`.
To further increase performance, we implement *blocking* (tiling) in all three loop dimensions. Instead of working on the full iteration space at once, we partition the indices into blocks of size BS along i, j, and k, and compute the product block-by-block:

```
void square_dgemm(int n, double* A, double* B, double* C)
{
  for (int jj = 0; jj < n; jj += BS) {
    int jmax = imin(jj + BS, n);

    for (int kk = 0; kk < n; kk += BS) {
      int kmax = imin(kk + BS, n);
```

```
      for (int ii = 0; ii < n; ii += BS) {
        int imax = imin(ii + BS, n);

        for (int j = jj; j < jmax; ++j) {
          for (int k = kk; k < kmax; ++k) {
            double bkj = B[k + j * n];
            for (int i = ii; i < imax; ++i) {
              C[i + j * n] += A[i + k * n] * bkj;
            }
          }
        }
      }
    }
  }
}
```

The outer loops `jj`, `kk`, and `ii` select the current block ranges

$$j \in [jj, j_{\max}), \qquad k \in [kk, k_{\max}), \qquad i \in [ii, i_{\max}),$$

and the helper `imin` ensures correct handling of boundary tiles when n is not a multiple of BS. Within each block triple, the inner kernel performs the same j–k–i update as in Task 1, but restricted to the current submatrices.

Blocking improves performance by increasing *temporal locality*. For a fixed (jj, kk, ii) triple, the algorithm repeatedly reuses the same sub-blocks of A and B while accumulating into the corresponding sub-block of C, reducing the frequency at which data must be fetched from main memory. Inside each block, the innermost loop still runs over i, so the access to $A(:, k)$ and $C(:, j)$ remains stride-1 in column-major format. Finally, loading `bkj = B(k,j)` once per (j, k) pair provides scalar reuse and avoids redundant loads in the innermost loop.

We set the default block size to $BS = 64$, a common baseline choice for a first cache-blocked DGEMM implementation. With double-precision elements, a 64×64 tile contains 64^2 entries and occupies $64^2 \cdot 8 \approx 32\,\text{KiB}$. The combined footprint of the three tiles (A, B, C) involved in the update is therefore about $3 \cdot 32 \approx 96\,\text{KiB}$. This working set does not fit in L1 data cache, but it fits comfortably in the per-core L2 cache, allowing substantial reuse from L2 rather than repeatedly streaming the same data from main memory. The block size is kept configurable via `#ifndef BS` so that alternative values can be tested empirically on the target architecture if further tuning is desired.

In figure 4, we show the performance of the naive, reordered (stride-one), blocked and BLAS implementations of DGEMM on a single core of an AMD EPYC 7763 processor. The naive i–j–k implementation stays around a few GFLOP/s and shows large variability (occasional deep drops), consistent with poor cache behavior due to strided accesses in column-major layout. Reordering the loops to achieve stride-one access improves locality substantially, reaching $\sim$10–15 GFLOP/s for moderate sizes; for larger n its performance gradually decreases as the working set no longer fits in the upper cache levels and execution becomes increasingly limited by memory traffic and reduced reuse. Cache blocking stabilizes performance around $\sim$10 GFLOP/s over the full range by increasing temporal locality and reusing sub-tiles of A and B while updating a tile of C. The reference BLAS implementation remains the fastest (tens of GFLOP/s), highlighting the impact of architecture-tuned optimizations such as packing, register blocking, vectorized micro-kernels, and prefetching that go beyond simple loop transformations.

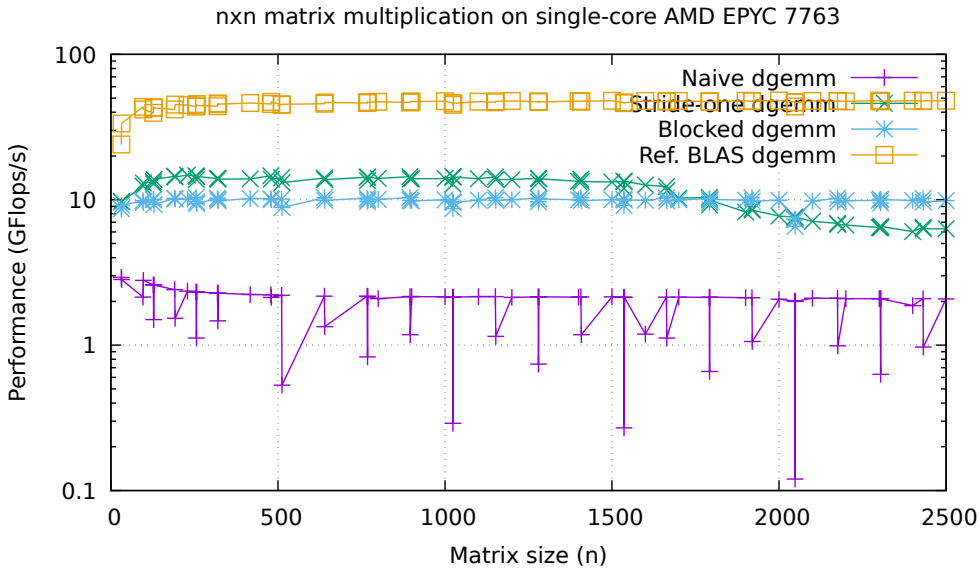


Figure 4: Performance results for the naive, reordered (stride-one), blocked and BLAS implementations of DGEMM (single core)